

Jenkins Docker Agent Setup Guide (Production)

Objective

Configure Jenkins (running inside Docker) to support Docker agents such as:

```
agent {  
  docker {  
    image 'python:3.12'  
    reuseNode true  
  }  
}
```

This enables Jenkins to dynamically pull Docker images (Python, Node.js, Java, Go, etc.) from Docker Hub and execute pipeline stages inside those containers.

Architecture

```
Ubuntu Server  
├── Docker Engine  
├── Jenkins Container  
│   ├── Jenkins  
│   ├── Docker CLI (mounted from host)  
│   ├── Git  
│   └── Jenkins Plugins  
├── Docker Socket  
│   ( /var/run/docker.sock )  
└── Temporary Docker Agent Containers  
    ├── python:3.12  
    ├── node:22  
    ├── eclipse-temurin:21  
    ├── golang:1.24  
    └── Any Docker image
```

Directory Structure

```
jenkins_compose/
├─ Dockerfile
├─ docker-compose.default.yml
├─ plugins.txt
└─ start.sh
```

Dockerfile

```
FROM jenkins/jenkins:lts

USER root

ARG DOCKER_GID

RUN apt-get update &&
    apt-get install -y
        git
        curl &&
    apt-get clean &&
    rm -rf /var/lib/apt/lists/*

# Create a group matching the host Docker socket GID
RUN groupadd -g ${DOCKER_GID} docker-host &&
    usermod -aG docker-host jenkins

USER jenkins

COPY plugins.txt /usr/share/jenkins/ref/plugins.txt

RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt
```

docker-compose.default.yml

```
services:
  jenkins:
    build:
      context: .
      dockerfile: Dockerfile
    args:
      DOCKER_GID: ${DOCKER_GID}
```

```
image: my-jenkins:lts
container_name: jenkins

ports:
  - "8080:8080"
  - "50000:50000"

environment:
  JAVA_OPTS: "-Xms512m -Xmx2048m -XX:+UseG1GC"

volumes:
  - jenkins_data:/var/jenkins_home
  - /var/run/docker.sock:/var/run/docker.sock
  - /usr/bin/docker:/usr/bin/docker:ro

restart: unless-stopped

volumes:
  jenkins_data:
```

plugins.txt

```
workflow-aggregator

git
git-client
github
github-api
github-branch-source
github-checks
pipeline-github

docker-workflow
docker-plugin
docker-commons

credentials
credentials-binding
plain-credentials
ssh-credentials
ssh-agent

pipeline-stage-view
pipeline-utility-steps
pipeline-input-step

job-dsl
```

```
copyartifact
rebuild

ws-cleanup
ansicolor
timestamper
build-timeout

junit
warnings-ng
htmlpublisher

email-ext
mailer

lockable-resources
throttle-concurrents

disk-usage

matrix-auth
role-strategy
authorize-project

configuration-as-code
```

start.sh

```
#!/bin/bash
set -e

echo "=====
echo " Jenkins Deployment"
echo "=====

export DOCKER_GID=$(stat -c '%g' /var/run/docker.sock)

echo "Docker Socket GID : ${DOCKER_GID}"

docker compose -f docker-compose.default.yml down

docker compose -f docker-compose.default.yml build --no-cache

docker compose -f docker-compose.default.yml up -d

echo
echo "=====
echo " Verification"
```

```
echo "===== "  
  
echo  
echo "Jenkins User Information:"  
docker exec -it jenkins id  
  
echo  
echo "Docker Access Test:"  
docker exec -it jenkins docker ps  
  
echo  
echo "===== "  
echo " Jenkins deployed successfully."  
echo "===== "
```

Make it executable:

```
chmod +x start.sh
```

Deploy Jenkins

Run a single command:

```
./start.sh
```

The script will:

1. Detect the Docker socket GID.
2. Export it as `DOCKER_GID`.
3. Build the custom Jenkins image.
4. Start Jenkins.
5. Verify Docker permissions.

Verification

The script automatically runs:

```
docker exec -it jenkins id
```

Expected output:

```
uid=1000(jenkins)
gid=1000(jenkins)
groups=1000(jenkins),988(docker-host)
```

It also runs:

```
docker exec -it jenkins docker ps
```

Expected output:

```
CONTAINER ID   IMAGE
xxxxxxxxxxxx   my-jenkins:lts
xxxxxxxxxxxx   nginx:alpine
xxxxxxxxxxxx   chatwave-app
...
```

If both commands succeed, Jenkins has permission to access the Docker daemon.

Example Jenkins Pipeline

```
pipeline {
    agent {
        docker {
            image 'python:3.12'
            reuseNode true
        }
    }

    stages {
        stage('Install Dependencies') {
            steps {
                sh '''
                    python -m venv venv
                    . venv/bin/activate

                    pip install -r requirements.txt
                '''
            }
        }

        stage('Run') {
            steps {
                sh '''
```

```
        . venv/bin/activate
        python get_vms.py
    ...
    }
}
}
```

Supported Docker Agent Images

Examples:

```
python:3.12
node:22
ubuntu:24.04
eclipse-temurin:21
maven:3.9.9-eclipse-temurin-21
golang:1.24
hashicorp/terraform
```

Each Jenkins pipeline can specify its own Docker image.

Benefits

- No hardcoded Docker socket GID.
 - Official Jenkins startup process remains unchanged.
 - Docker socket permissions are configured during image build.
 - Docker agents work for Python, Node.js, Java, Go, Terraform, and other Docker images.
 - Clean, isolated build environment for every pipeline.
 - Easy to rebuild and redeploy using a single script.
 - Jenkins configuration and plugins are reproducible through code.
-

Conclusion

With this setup, Jenkins acts as the CI/CD controller, while Docker provides isolated build environments. Pipelines can use any suitable Docker image as an agent without installing those runtimes directly into the Jenkins container, resulting in a clean, maintainable, and production-ready deployment.